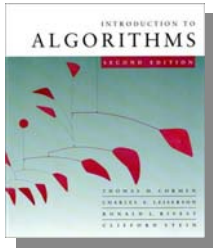


Fibonacci numbers

Recursive definition:

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2. \end{cases}$$

0 1 1 2 3 5 8 13 21 34 ...



Fibonacci numbers

Recursive definition:

$$F_n = \begin{cases} 0 & \text{if } n = 0; \\ 1 & \text{if } n = 1; \\ F_{n-1} + F_{n-2} & \text{if } n \geq 2. \end{cases}$$

0 1 1 2 3 5 8 13 21 34 ...

Naive recursive algorithm: $\Omega(\phi^n)$
(exponential time), where $\phi = (1 + \sqrt{5})/2$
is the *golden ratio*.

Golden ratio $\phi = \frac{1 + \sqrt{5}}{2}$

for any two numbers a and b $a > b > 0$

$$\phi = \frac{a+b}{a} = \frac{a}{b} \Rightarrow \text{quadratic in } \phi.$$

Relation to Fibonacci numbers:

$$\lim_{n \rightarrow \infty} \frac{F_n}{F_{n-1}} = \phi$$

ϕ as continued fraction, $\phi = 1 + \frac{1}{1 + \frac{1}{1 + \frac{1}{1 + \frac{1}{1 + \dots \infty}}}}$

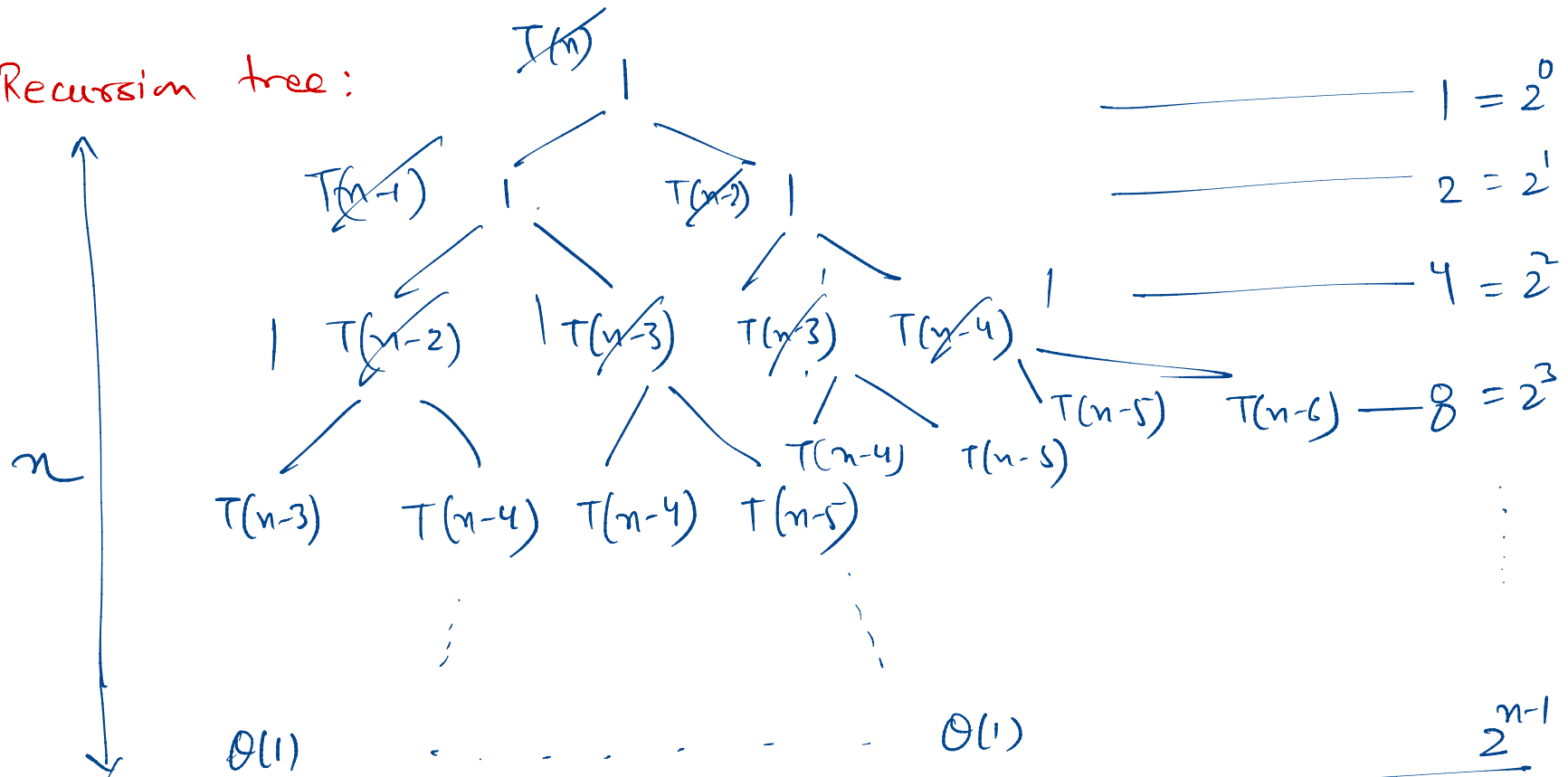
Naive computation of Fibonacci numbers.

$$F_n = F_{n-1} + F_{n-2}$$

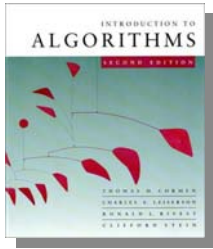
Open MIP
for multithreading

$$T(n) = T(n-1) + T(n-2) + \Theta(1) \quad \text{--- Rec relation}$$

Recursion tree:



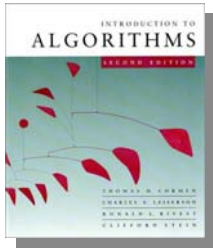
$$T(n) = \sum_{i=1}^n 2^{i-1} = 2^n = \boxed{O(2^n)}$$



Computing Fibonacci numbers

Bottom-up:

- Compute $F_0, F_1, F_2, \dots, F_n$ in order, forming each number by summing the two previous.
- Running time: $\Theta(n)$.



Computing Fibonacci numbers

Bottom-up:

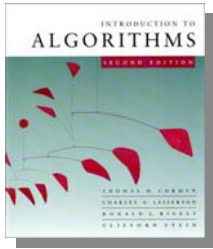
- Compute $F_0, F_1, F_2, \dots, F_n$ in order, forming each number by summing the two previous.
- Running time: $\Theta(n)$.

Naive recursive squaring:

Binet's formula

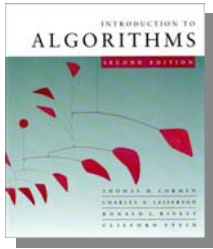
$F_n = \phi^n / \sqrt{5}$ rounded to the nearest integer.

- Recursive squaring: $\Theta(\lg n)$ time.
- This method is unreliable, since floating-point arithmetic is prone to round-off errors.



Recursive squaring

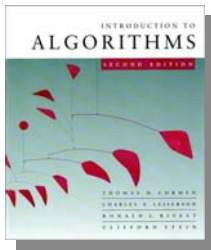
Theorem:
$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n.$$



Recursive squaring

Theorem:
$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n .$$

Algorithm: Recursive squaring.
Time = $\Theta(\lg n)$.



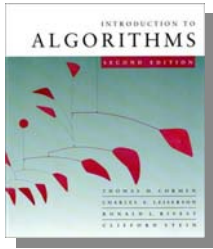
Recursive squaring

Theorem:
$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n.$$

Algorithm: Recursive squaring.
Time = $\Theta(\lg n)$.

Proof of theorem. (Induction on n .)

Base ($n = 1$):
$$\begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^1.$$

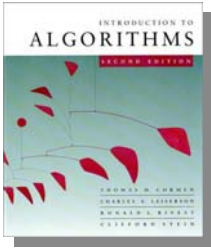


Recursive squaring

Inductive step ($n \geq 2$):

$$\begin{aligned}
 \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} &= \begin{bmatrix} F_n & F_{n-1} \\ F_{n-1} & F_{n-2} \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \cdot \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n
 \end{aligned}$$

Handwritten annotations: A blue arrow points from the inductive step text to the matrix equation. To the right, a diagram shows the recursive squaring process. It features a large blue bracket containing two rows: $(F_n + F_{n-1})$ and $(F_{n-1} + F_{n-2})$. Red arrows point from these expressions to F_{n+1} and F_n respectively. To the right of this bracket, another blue bracket contains the terms F_n and F_{n-1} .



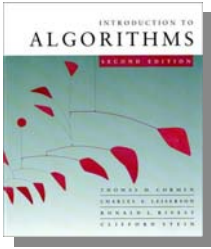
Matrix multiplication

Input: $A = [a_{ij}], B = [b_{ij}].$ } $i, j = 1, 2, \dots, n.$
Output: $C = [c_{ij}] = A \cdot B.$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

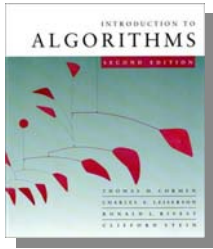
Handwritten annotations:
 - A red circle around c_{11} with an arrow pointing to it from the text "dot product = row^A • col^B".
 - A red box around the first row of matrix A with an arrow pointing to it from the text "row^A".
 - A red box around the first column of matrix B with an arrow pointing to it from the text "col^B".

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$



Standard algorithm

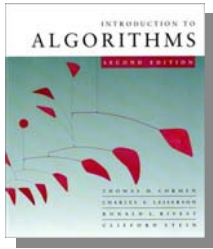
```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$ 
```



Standard algorithm

```
for  $i \leftarrow 1$  to  $n$ 
  do for  $j \leftarrow 1$  to  $n$ 
    do  $c_{ij} \leftarrow 0$ 
      for  $k \leftarrow 1$  to  $n$ 
        do  $c_{ij} \leftarrow c_{ij} + a_{ik} \cdot b_{kj}$ 
```

Running time = $\Theta(n^3)$



Divide-and-conquer algorithm

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

Cost of addition

$$4 \left(\left[\frac{n^2}{4} \right]_{n/2 \times n/2} + \left[\frac{n^2}{4} \right]_{n/2 \times n/2} \right)$$

block $\rightarrow$

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

Block multiplication

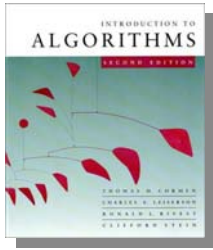
$$C = A \cdot B$$

$$\left. \begin{aligned} r &= ae + bg \\ s &= af + bh \\ t &= ce + dg \\ u &= cf + dh \end{aligned} \right\}$$

$$T(n) = \underbrace{8 T(n/2)}_{\text{cost of multiplication}} + \theta(n^2)$$

8 mults of $(n/2) \times (n/2)$ submatrices

4 adds of $(n/2) \times (n/2)$ submatrices



Divide-and-conquer algorithm

IDEA:

$n \times n$ matrix = 2×2 matrix of $(n/2) \times (n/2)$ submatrices:

$$\begin{bmatrix} r & s \\ t & u \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix}$$

$$C = A \cdot B$$

$$r = ae + bg$$

$$s = af + bh$$

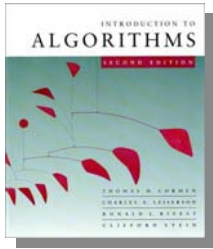
$$t = ce + dh$$

$$u = cf + dg$$

recursive

8 mults of $(n/2) \times (n/2)$ submatrices

4 adds of $(n/2) \times (n/2)$ submatrices



Analysis of D&C algorithm

$$T(n) = a T(n/b) + f(n)$$

$$T(n) = 8T(n/2) + \Theta(n^2)$$

$$a = 8, b = 2$$

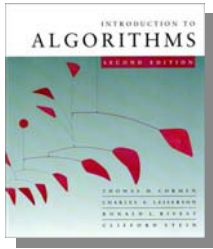
$$f(n) = n^2$$

$$n^{\log_b a} = n^3$$

submatrices

submatrix size

work adding
submatrices



Analysis of D&C algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

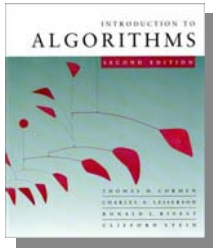
submatrices

submatrix size

work adding
submatrices

$$n^{\log_b a} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow \underline{T(n) = \Theta(n^3)}.$$

$$T(n) = 8T(n/2) + \Theta(n^2)$$



Analysis of D&C algorithm

$$T(n) = 8T(n/2) + \Theta(n^2)$$

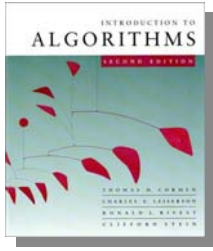
submatrices

submatrix size

work adding
submatrices

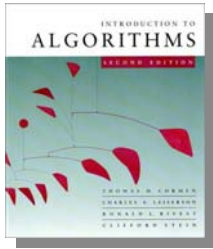
$$n^{\log_b a} = n^{\log_2 8} = n^3 \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^3).$$

No better than the ordinary algorithm.



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

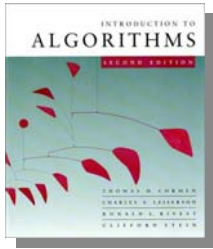
$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

7 matrix mults

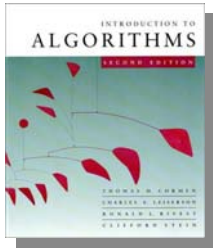
$$\begin{aligned} P_1 &= a \cdot (f - h) \\ P_2 &= (a + b) \cdot h \\ P_3 &= (c + d) \cdot e \\ P_4 &= d \cdot (g - e) \\ P_5 &= (a + d) \cdot (e + h) \\ P_6 &= (b - d) \cdot (g + h) \\ P_7 &= (a - c) \cdot (e + f) \end{aligned}$$

$$r = P_5 + P_4 - P_2 + P_6$$

$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

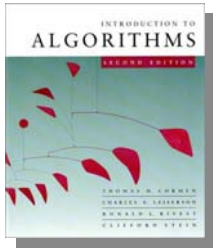
$$s = P_1 + P_2$$

$$t = P_3 + P_4$$

$$u = P_5 + P_1 - P_3 - P_7$$

7 mults, 18 adds/subs.

Note: No reliance on commutativity of mult!



Strassen's idea

- Multiply 2×2 matrices with only 7 recursive mults.

$$P_1 = a \cdot (f - h)$$

$$P_2 = (a + b) \cdot h$$

$$P_3 = (c + d) \cdot e$$

$$P_4 = d \cdot (g - e)$$

$$P_5 = (a + d) \cdot (e + h)$$

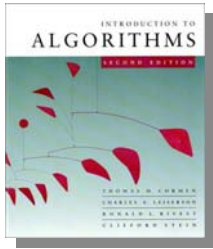
$$P_6 = (b - d) \cdot (g + h)$$

$$P_7 = (a - c) \cdot (e + f)$$

$$r = P_5 + P_4 - P_2 + P_6$$

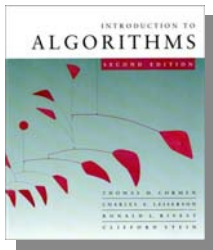
$$\begin{aligned} &= (a + d)(e + h) \\ &\quad + d(g - e) - (a + b)h \\ &\quad + (b - d)(g + h) \end{aligned}$$

$$\begin{aligned} &= ae + ah + de + dh \\ &\quad + dg - de - ah - bh \\ &\quad + bg + bh - dg - dh \\ &= ae + bg \end{aligned}$$



Strassen's algorithm

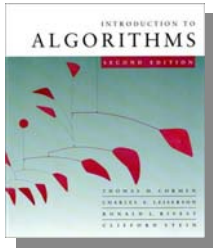
1. **Divide:** Partition A and B into $(n/2) \times (n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
2. **Conquer:** Perform 7 multiplications of $(n/2) \times (n/2)$ submatrices recursively.
3. **Combine:** Form C using $+$ and $-$ on $(n/2) \times (n/2)$ submatrices.



Strassen's algorithm

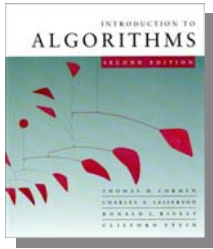
1. **Divide:** Partition A and B into $(n/2) \times (n/2)$ submatrices. Form terms to be multiplied using $+$ and $-$.
2. **Conquer:** Perform 7 multiplications of $(n/2) \times (n/2)$ submatrices recursively.
3. **Combine:** Form C using $+$ and $-$ on $(n/2) \times (n/2)$ submatrices.

$$T(n) = 7 T(n/2) + \Theta(n^2)$$



Analysis of Strassen

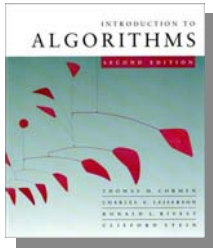
$$T(n) = 7 T(n/2) + \Theta(n^2)$$



Analysis of Strassen

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\lg 7}).$$

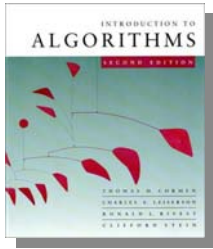


Analysis of Strassen

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.



Analysis of Strassen

$$T(n) = 7 T(n/2) + \Theta(n^2)$$

$$n^{\log_b a} = n^{\log_2 7} \approx n^{2.81} \Rightarrow \text{CASE 1} \Rightarrow T(n) = \Theta(n^{\lg 7}).$$

The number 2.81 may not seem much smaller than 3, but because the difference is in the exponent, the impact on running time is significant. In fact, Strassen's algorithm beats the ordinary algorithm on today's machines for $n \geq 32$ or so.

Best to date (of theoretical interest only): $\Theta(n^{2.3713})$ (2024) $n^{2.3713}$
Best to date (of theoretical interest only): $\Theta(n^{2.376\dots})$.

Tower of Hanoi

- A puzzle containing 3 rods and a number of discs of various sizes (diameters) which can slide onto any rod.
- The puzzle begins with all discs stacked on one rod in order of decreasing size; smallest on top.

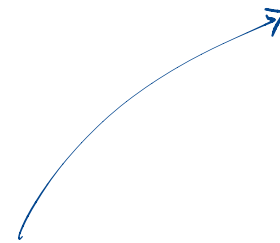
Objective : Move the entire stack to one of the other empty rods.

- Rules :**
1. Only 1 disc can be moved at a time
 2. discs can only be moved from/to top of a stack or on an empty rod.
 3. Only a smaller disc may be placed on top of an existing disc.

# discs	# steps
$n=1$	1
$n=2$	3
$n=3$	7
$n=4$	15
$\vdots$	
$n=k$	$2^k - 1$

Complexity

$$T(n) = 2^n - 1$$

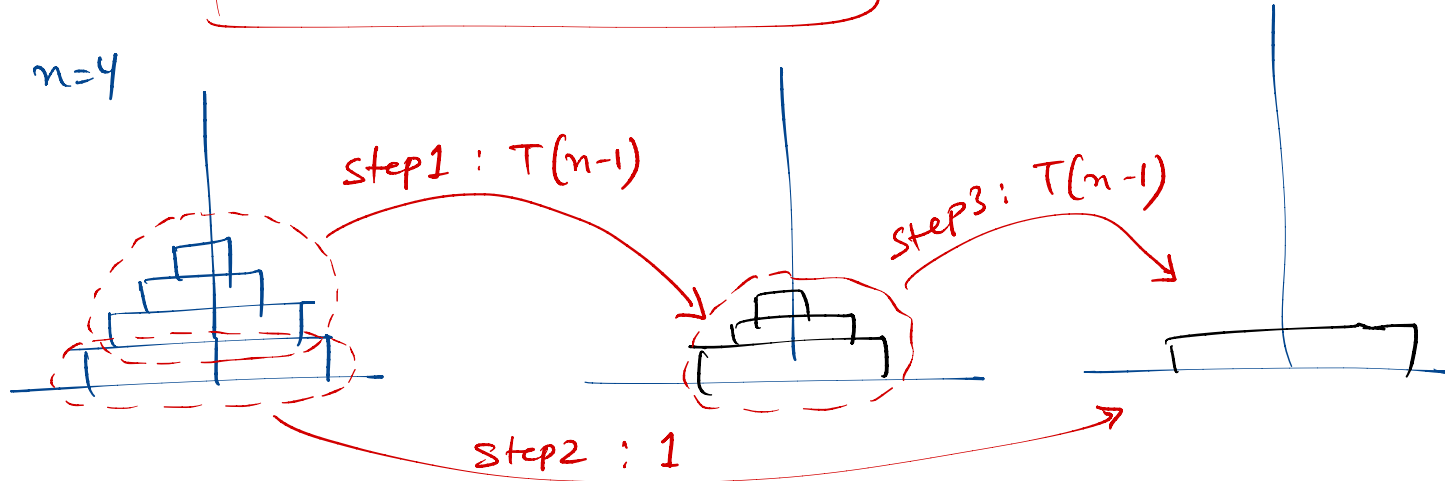


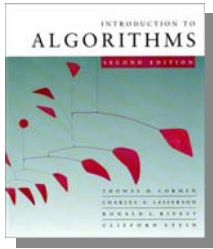
$$\Rightarrow T(n) = \Theta(2^n)$$

Recursion:

$$T(n) = 2T(n-1) + 1$$

lets take $n=4$





Conclusion

- Divide and conquer is just one of several powerful techniques for algorithm design.
- Divide-and-conquer algorithms can be analyzed using recurrences and the master method (so practice this math).
- The divide-and-conquer strategy often leads to efficient algorithms.